

SPORTTRACKER

Feuille de Route Stratégique & Plan de Développement

Objectif : MVP de Suivi de Tournois Sportifs (Focus initial : Football)
Cible : Usage Personnel & Cercle Proche (Famille/Amis)
Valorisation : Portfolio Technique - Recherche de Stage M1

1. Alignement des Objectifs & Vision

Ce projet vise à concevoir une architecture logicielle robuste, scalable et économiquement viable (gratuite en phase initiale) pour suivre l'évolution globale des compétitions sportives. Contrairement aux outils de "live-scoring" classiques, SportTracker se positionne sur le suivi macro des événements : notification dès la fin des rencontres, mise à jour des arbres de tournois et des classements de phases de poules.

Contrainte Majeure & Solution Technique

Pour respecter la gratuité et contourner les limites strictes des API de sport gratuites (ex: 100 requêtes/jour), le projet implémente une stratégie de **Smart Polling** via une file d'attente de tâches asynchrones. Le serveur est le seul point de contact avec l'API externe, protégeant ainsi les quotas contre l'augmentation du nombre d'utilisateurs.

2. Architecture Technique Cible

- **Frontend Web (MVP) :** **React.js** pour une interface dynamique, modulaire et hautement réutilisable.
- **Application Mobile (Cible) :** **React Native** permettant de porter la logique métier web sur mobile à 90%.
- **Backend & API :** **Node.js / Express** (ou Python) pour orchestrer les routes de l'application et la logique métier.
- **Base de Données :** **PostgreSQL** (ou MongoDB) pour stocker localement les données "à froid" des compétitions.
- **Gestionnaire de Tâches :** **BullMQ / Redis** (ou Celery) pour piloter le Smart Polling asynchrone des matchs.

3. Phases de Développement Détaillées

Phase 1 : Conception, Modélisation & Maquettage

Semaine 1

Initialisation des bases théoriques et visuelles du projet afin d'éviter toute dette technique lors du codage.

Livrables clés :

- ☐ Modélisation de la base de données (Schéma Entité-Association : Tournois, Saisons, Équipes, Matches).
- ☐ Spécification des endpoints de notre propre API REST.
- ☐ Création des wireframes (Figma ou papier) des écrans clés : Tableau de bord, Arbre du tournoi, Profil des favoris.

Phase 2 : Backend, Base de Données & Moteur d'Automatisation

Semaines 2 - 3

Mise en place du cerveau de l'application, de la persistance des données et du script d'actualisation intelligente.

Livrables clés :

- ☐ Initialisation de la base de données et scripts de migration.
- ☐ Développement du script de "Routine Matinale" (récupération quotidienne des matchs du jour à minuit).
- ☐ Développement de l'algorithme de **Smart Polling** (planification d'une vérification à T+105min du début d'un match, puis boucle de re-vérification toutes les 5min jusqu'au statut final).
- ☐ Création et sécurisation des routes API pour le Frontend.

Phase 3 : Frontend Web & Déploiement du MVP

Semaines 4 - 5

Création de l'interface utilisateur web pour permettre à l'entourage de tester l'outil et valider l'expérience.

Livrables clés :

- ☐ Mise en place de l'environnement React (Vite.js) et structure de dossiers propre.
- ☐ Développement des composants réutilisables (Cartes de match, Arbre de tournoi dynamique).
- ☐ Consommation des données de notre API Backend via Axios ou Fetch.
- ☐ Déploiement du Backend (ex: Render/Railway) et du Frontend (ex: Vercel/Netlify) sur des plans gratuits.

Portage de la solution sur smartphone pour offrir une vraie expérience applicative aux proches.

Livrables clés :

- ☐ Initialisation du projet React Native (Expo pour faciliter le partage).
- ☐ Migration des hooks de logique et des services d'appels API du projet Web.
- ☐ Adaptation des composants graphiques pour le mobile (changement des balises HTML en composants natifs).
- ☐ Génération de l'APK (Android) et distribution en mode ad-hoc pour les tests.

4. Critères de Qualité Professionnelle (Atouts Stage M1)

Pour maximiser l'impact de ce projet auprès des recruteurs lors de la recherche de stage, les standards professionnels suivants seront rigoureusement appliqués :

- **Git Flow Strict** : Utilisation de branches dédiées (feature/, bugfix/, main) et messages de commit explicites (norme Conventional Commits).
- **Gestion des Erreurs** : Implémentation de mécanismes de repli (Fallback) si l'API externe subit une panne ou si les quotas sont dépassés, afin de garantir la continuité de service du Frontend.
- **Documentation (README)** : Rédaction d'un guide complet expliquant l'architecture logicielle, les choix de conception (justification du Smart Polling) et la procédure d'installation locale.